UNITY 6000.0.62F1 · COMPLETE OFFLINE DOCUMENTATION

Rope Riddle Developer Guide

Install. Edit. Reskin. Build your release.

szekipapa77@gmail.com



CHAPTER 01

Start here: Rope Riddle

Rope Riddle is a standalone editable Unity game template. Choose letters on the onscreen keyboard, or type them, to uncover the hidden word. Wrong guesses cost rope strands. Solve the word before the available strands run out; helper effects change letters or the remaining mistake allowance.

Includes 25 curated levels, source-backed generation, a game-specific visual editor and validation, 4 guided helpers, local saving and portrait/landscape scenes. Keep the required private game Kit and publisher-owned Szekipapa77Ads layer. Provider SDK binaries are excluded. There is no bundled HLK, purchases, multiplayer server or remote LiveOps backend. Import only one game Kit into a project because the Kit classes share names.

CHAPTER 02

Requirements, installation and first play

Use Unity **6000.0.62f1**, URP **17.0.4** and Unity UI **2.0.0**. Install matching Android SDK/NDK/OpenJDK and WebGL modules in Unity Hub. Retain the supplied manifest and ProjectSettings.

1. Extract the standalone project to a short writable local path, then open it in the matching editor. Wait for package import and compilation.
2. For native unitypackage import, create a matching URP project and use Assets > Import Package > Custom Package. Import every game, Kit, Szekipapa77Ads and WebGL-template entry.
3. Run Szekipapa77 > Rope Riddle > Setup. It configures rendering and player settings; review its changes when integrating with existing work.
4. Active Input Handling must support the classic Input Manager: choose Old or Both. New Input System only is insufficient.
5. Open `Assets/Games/RopeWord/Scenes/RopeWord_Mobile.unity` and press Play. Follow the level card and first-level instructions.
6. Stop Play mode before editing level assets. Keep all .meta GUIDs. The game uses classic Unity UI Text; do not assume an extra TextMesh Pro import is required.

CHAPTER 03

Project folders and exact source map

All game-relative paths below start at `Assets/Games/RopeWord`. Runtime UI is built in code; editing an unrelated static Canvas does not reskin it. Scripts contains game rules and views; Editor contains authoring tools; Kit supplies UI/audio/save/platform support; Resources/ropeword contains level JSON, art, music and sound; Settings owns URP assets; Scenes contains both entry scenes.

- `Scripts/RopeArt.cs`
- `Scripts/RopeLogic.cs`
- `Scripts/RopeWordGame.cs`
- `Scripts/RopeWordGame.UI.cs`
- `Scripts/RopeWordGame.View.cs`
- `Scripts/RopeWords.cs`

Main flow: RopeWordGame.cs. UI and popup consumers: RopeWordGame.UI.cs. Model/generator/loading: RopeLogic.cs. Authoring: Editor/RopeWordLevelEditor.cs. Shared ad settings live outside the game at Assets/Szekipapa77Ads/Resources/AdsSettings.asset.

CHAPTER 04

Gameplay, controls and progression

Choose letters on the onscreen keyboard, or type them, to uncover the hidden word. Wrong guesses cost rope strands. Solve the word before the available strands run out; helper effects change letters or the remaining mistake allowance.

The initial bank has 25 curated levels. Read the supplied loader/generator before changing continuation rules. Play the guided opening, ordinary progress, completion and retry paths; preserve saved indexes during a visual reskin. The Mobile scene is portrait and the WebGL-named scene is landscape, but the supplied Build menu selects portrait for both build targets. Select landscape explicitly in your Build Profile if desired. Test a near-startup scene replacement as well as a normal restart.

CHAPTER 05

Tutorials and every helper

Fresh local progress triggers the first-level guidance. Newly unlocked helpers have introduction and guided-use flows. Reset only a test save when reviewing tutorials. A free guided first use can differ from ordinary charge consumption. Exact helper names/unlocks/charges: Scripts/RopeWordGame.cs; help and effect descriptions: Scripts/RopeWordGame.cs.

Helper	Unlock	Free charges	Action and effect
Reveal	3	3	Uncovers the first hidden letter of the answer.
Snip	6	2	Cuts 3 wrong letters off the keyboard. No strands lost!
Lantern	10	2	Lights up the vowels: the right ones appear, the wrong ones go dark.
Spare Rope	15	1	Ties one extra strand onto the rope: one more mistake allowed.

Activate the tool and perform its required target action: arming a tool alone does not demonstrate its effect. A successful use requires the stated preconditions from this game: show both activation and its actual effect. Exhausted charges may offer a rewarded ad; verify success/cancel/failure and grant a reward once only.

CHAPTER 06

Visual level editor: worked editing session

Open **Szekipapa77 > Rope Riddle > Level Editor**. Exact implementation: `Editor/RopeWordLevelEditor.cs`.

Duplicate an early entry in this game-specific visual editor, change its visible name and one rule, save, validate, then play it. Use the exact source-control table below for this game.

1. Back up Resources/ropeword/Levels/levels.json.
2. Duplicate a simple early level and edit one small element.
3. Use the game-specific validation controls and resolve every reported issue.
4. Save explicitly; stop and restart Play mode, then use Play this level or navigate to the edited index.
5. Check portrait and landscape and rebuild demo outputs after accepting data changes.

The exact current authoring controls, including tooltips and generator switches, follow. These are game-specific source references, not a generic editor feature list.

Source	Exact current code
`RopeWordLevelEditor.cs:47`	`[MenuItem("Szekipapa77/Rope Riddle/Level Editor")]`
`RopeWordLevelEditor.cs:50`	`[MenuItem("Szekipapa77/Rope Riddle/Validate All Levels")]`
`RopeWordLevelEditor.cs:120`	`if (GUILayout.Button(_dirty ? "Save *" : "Save", EditorStyles.toolbarButton, GUILayout.Width(70))) Save();`
`RopeWordLevelEditor.cs:121`	`if (GUILayout.Button("Reload", EditorStyles.toolbarButton, GUILayout.Width(60)) && (!_dirty || EditorUtility.DisplayDialog("Reload", "Discard unsaved changes?", "Discard", "Cancel"))) Load();`
`RopeWordLevelEditor.cs:124`	`if (GUILayout.Button("Validate", EditorStyles.toolbarButton, GUILayout.Width(70)) && Cur != null) ValidateOne(_sel);`
`RopeWordLevelEditor.cs:125`	`if (GUILayout.Button("Validate All", EditorStyles.toolbarButton, GUILayout.Width(85))) ValidateAll();`

RopeWordLevelEditor.cs:128	`if (GUILayout.Button(new GUIContent("Quick test", "Optional: saves, opens the WebGL scene at this level and enters Play mode. Editing never needs it."), EditorStyles.toolbarButton, GUILayout.Width(80)) && Cur != null) Play(_sel + 1);`
RopeWordLevelEditor.cs:155	`if (GUILayout.Button("Add"))`
RopeWordLevelEditor.cs:161	`if (GUILayout.Button("Dup") && Cur != null) Insert(_sel + 1, Cur.Clone());`
RopeWordLevelEditor.cs:162	`if (GUILayout.Button("Del") && Cur != null && EditorUtility.DisplayDialog("Delete", $"Delete level {_sel + 1} '{Cur.answer}'?", "Delete", "Cancel"))`
RopeWordLevelEditor.cs:166	`if (GUILayout.Button("Up") && _sel > 0) Move(-1);`
RopeWordLevelEditor.cs:167	`if (GUILayout.Button("Down") && _sel < _file.levels.Length - 1) Move(1);`
RopeWordLevelEditor.cs:169	`EditorGUILayout.LabelField("Tags: B breather, 2W two words, T timed, ? mystery clue, sN strands", EditorStyles.miniLabel);`
RopeWordLevelEditor.cs:191	`EditorGUILayout.LabelField($"Level {_sel + 1}", EditorStyles.boldLabel);`
RopeWordLevelEditor.cs:197	`_answerEdit = EditorGUILayout.TextField("Answer", _answerEdit);`
RopeWordLevelEditor.cs:205	`if (GUILayout.Button("Pick from bank", GUILayout.Width(110))) BankMenu(l);`
RopeWordLevelEditor.cs:208	`l.category = EditorGUILayout.TextField("Category (clue)", l.category);`
RopeWordLevelEditor.cs:209	`if (GUILayout.Button("Categories", GUILayout.Width(110)))`
RopeWordLevelEditor.cs:218	`l.strands = EditorGUILayout.IntSlider("Strands (misses allowed)", l.strands, RopeRules.MinStrands, RopeRules.MaxStrands);`
RopeWordLevelEditor.cs:219	`bool timed = EditorGUILayout.ToggleLeft("Timed bonus", l.timer > 0, GUILayout.Width(100));`

RopeWordLevelEditor.cs:225	l.hidden = EditorGUILayout.ToggleLeft("Mystery clue (hidden category)", l.hidden, GUILayout.Width(220));
RopeWordLevelEditor.cs:226	l.breather = EditorGUILayout.ToggleLeft("Breather", l.breather, GUILayout.Width(90));
RopeWordLevelEditor.cs:228	si = EditorGUILayout.Popup("Scene", si, RopeBank.Scenes);
RopeWordLevelEditor.cs:231	l.hint = EditorGUILayout.TextField("First-time hint", l.hint);
RopeWordLevelEditor.cs:239	EditorGUILayout.HelpBox($"Solver bot: {v}\nStored difficulty {l.difficulty:0.00} (Validate writes score/100).", type);
RopeWordLevelEditor.cs:269	if (problems.Count > 0) EditorGUILayout.HelpBox("Word check: " + string.Join("; ", problems), fmt != null ? MessageType.Error : MessageType.Warning);
RopeWordLevelEditor.cs:284	EditorGUILayout.LabelField("Preview: click a letter slot to pre-reveal it, a pip to set the strands, a thumbnail to change the scene.", EditorStyles.miniLabel);
RopeWordLevelEditor.cs:391	_showCurve = EditorGUILayout.Foldout(_showCurve, "Difficulty curve (bot score per level; blue = breather, red outline = unfair)", true);
RopeWordLevelEditor.cs:422	_showGen = EditorGUILayout.Foldout(_showGen, "Auto Generate (seeded, bot-validated, the same RopeGen the game uses after the curated list)", true);
RopeWordLevelEditor.cs:425	_genCount = Mathf.Clamp(EditorGUILayout.IntField("Count", _genCount), 1, 200); _genStart = Mathf.Max(1, EditorGUILayout.IntField("Start at level #", _genStart));
RopeWordLevelEditor.cs:426	_genSeed = EditorGUILayout.IntField("Seed", _genSeed);

`RopeWordLevelEditor.cs:429`	`_genFrom = EditorGUILayout.Slider("Start difficulty", _genFrom, 0, 1); _genTo = EditorGUILayout.Slider("End difficulty", _genTo, 0, 1);`
`RopeWordLevelEditor.cs:432`	`_genBreather = EditorGUILayout.IntSlider("Breather every", _genBreather, 0, 8);`
`RopeWordLevelEditor.cs:433`	`_genReplace = EditorGUILayout.ToggleLeft("Replace that range (else insert)", _genReplace);`
`RopeWordLevelEditor.cs:444`	`EditorGUILayout.LabelField($"Mechanics unlock by level number: two-word phrases from {RopeGen.PhraseFrom}, fewer strands from {RopeGen.FewerFrom}, timed from {RopeGen.TimedFrom}, mystery clue from {RopeGen.HiddenFrom}.", EditorStyles.wordWrappedMiniLabel);`
`RopeWordLevelEditor.cs:445`	`if (GUILayout.Button($"Generate {_genCount} level(s)", GUILayout.Height(26))) Generate();`

CHAPTER 07

Generator, insertion and replacement

Use this editor generation panel, choose a reproducible seed and small count, review start/index and replace behavior, then generate, save and reopen. The source excerpt below identifies exact append/replace semantics.

Source	Exact current code
`RopeWordLevelEditor.cs:44`	`bool _genReplace, _showGen = true, _showCurve = true;`
`RopeWordLevelEditor.cs:433`	`_genReplace = EditorGUILayout.ToggleLeft("Replace that range (else insert)", _genReplace);`
`RopeWordLevelEditor.cs:452`	`if (_genReplace) { int rm = Mathf.Min(_genCount, list.Count - at); if (rm > 0) list.RemoveRange(at, rm); }`
`RopeWordLevelEditor.cs:462`	`list.InsertRange(at, made);`

Worked extension: back up the bank, generate a small set at moderate difficulty using the exact per-game controls, validate them, then Save. Reopen the JSON and confirm the count changed. Do not assume every editor has the same insertion behavior: the actual implementation below distinguishes insert, append and replace. Runtime continuation does not automatically append new records to project JSON. Validation and bot scores help find problems but do not certify a pleasant human difficulty curve.

CHAPTER 08

Actual level JSON and identifiers

Canonical asset: `Assets/Games/RopeWord/Resources/ropeword/Levels/levels.json`. Array order controls visible numbering. Preserve numeric/boolean types, root version fields where present and resource/save identifiers. The actual first record is shown below; use the editor for structured coordinates and block/word data.


```
{
  "answer": "train",
  "category": "Vehicles",
  "strands": 7,
  "timer": 0,
  "hidden": false,
  "shown": "",
  "scene": "canyon",
  "breather": false,
  "hint": "",
  "difficulty": 0.21
}
```


Root keys: levels. Inspect RopeLogic.cs for field semantics and validation. Keep visible names/hints distinct from technical keys. The game-specific board, wave, card, opponent or question records are gameplay data; UI translation is separate.

CHAPTER 09

Reskin recipe: interface colours

Stop Play mode and open `Scripts/RopeWordGame.View.cs`. Find `Ink`, currently `#23384A`. Change that hex literal to `#42C9B0`, preserve its symbol/type, compile and reopen the scene. Review HUD, menus, helper cards, settings/results and both orientations. Art sprites and procedural fallback shapes can have additional colours; edit them deliberately.

Actual consumers of this symbol are listed first, then all current palette/hex assignments. Editor-only palettes do not control the runtime. Preserve state/goal contrast and rebuild after accepting a change.

Source	Exact current code
`RopeWordGame.cs:265 / void OnHelper(int h)`	`Bubble(d.Name + " unlocks at level " + d.Unlock, Ink, 1.5f, HelperRect(h));`
`RopeWordGame.cs:270 / void OnHelper(int h)`	`if (!CanUse(h, out var why)) { HzAudio.Play("locked", 0.5f); HzTween.Shake(_helperUI[h].rt, 8, 0.25f); Bubble(why, Ink, 1.4f, HelperRect(h)); return; }`
`RopeWordGame.UI.cs:58 / void LevelHint()`	`CardText(m, _data.hint, 290, 720, 150, 40, Ink);`
`RopeWordGame.UI.cs:87 / void ShowUnlock(List<int> list, int k)`	`CardText(m, d.Name, 540, 700, 90, 70, Ink, true);`
`RopeWordGame.UI.cs:127 / void ShowGuide(int h)`	`Bubble(Defs[h].Guide, Ink, 0, r, _guideLayer);`
`RopeWordGame.UI.cs:299 / void ShowWin(int level, int stars, int gems, bool timedBonus)`	`CardText(m, R.ClueVisible ? R.category : "", 535, 760, 44, 34, InkSoft);`
`RopeWordGame.UI.cs:302 / void ShowWin(int level, int stars, int gems, bool timedBonus)`	`CardText(m, sub, 690, 780, 52, 40, Ink);`
`RopeWordGame.UI.cs:321 / void ShowLose()`	`CardText(m, "The rope gave way! Your progress:", 420, 780, 50, 36, InkSoft);`
`RopeWordGame.UI.cs:323 / void ShowLose()`	`CardText(m, "Level " + Level + " is waiting for another try.", 580, 780, 50, 36, Ink);`
`RopeWordGame.UI.cs:350 / void OfferGems(int cost)`	`CardText(m, $"This helper costs {cost} gems.\nEvery cleared level pays gems too.", 300, 700, 120, 36, Ink);`
`RopeWordGame.UI.cs:364 / void OfferGems(int cost)`	`CardButton(m, "No, thanks", InkSoft, 570, 420, 96, () => CloseModal(m), 42);`

RopeWordGame.UI.cs:440 / void ShowMenu()	var t1 = HzUI.Label("T", sign.transform, "Rope Riddle", portrait ? 120 : 130, Ink); t1.rectTransform.Stretch(); t1.rectTransform.anchoredPosition = new Vector2(0, 18);
RopeWordGame.UI.cs:442 / void ShowMenu()	var t2 = HzUI.Label("S", sign.transform, "Guess the word. Keep the climber on the rope!", 34, InkSoft, TextAnchor.MiddleCenter, false);
RopeWordGame.UI.cs:455 / void ShowMenu()	HzUI.Label("Lv", lvTag.transform, "Level " + Level, 44, Ink).rectTransform.Stretch();
RopeWordGame.UI.cs:462 / void ShowMenu()	var sT = HzUI.Label("V", sp.transform, TotalStars.ToString(), 54, Ink); sT.rectTransform.Stretch(); sT.rectTransform.offsetMin = new Vector2(90, 0);
RopeWordGame.UI.cs:529 / void Dim(float x, float y, float w, float h) { if (w <= 0 || h <= 0) return; var d = HzUI.Panel("Dim", _tut, dark, 0); d.rectTransform.Place(new Vector2(0, 1), new Vector2(w, h), new Vector2(x, -y), new Vector2(0, 1)); }	var ct = HzUI.Label("T", cap.transform, TutText[_tutStep], 42, Ink); ct.rectTransform.Stretch(26);
RopeWordGame.UI.cs:530 / void Dim(float x, float y, float w, float h) { if (w <= 0 || h <= 0) return; var d = HzUI.Panel("Dim", _tut, dark, 0); d.rectTransform.Place(new Vector2(0, 1), new Vector2(w, h), new Vector2(x, -y), new Vector2(0, 1)); }	var step = HzUI.Label("N", cap.transform, (_tutStep + 1) + "/" + TutText.Length, 26, InkSoft, TextAnchor.UpperRight, false);
RopeWordGame.View.cs:18 / class/static declarations	static readonly Color Card = HzUI.Hex("#FFF7EA"), Ink = HzUI.Hex("#23384A"), InkSoft = HzUI.Hex("#6B7C8A");
RopeWordGame.View.cs:111 / void BuildView()	_levelTxt = HzUI.Label("Lv", lvPill.transform, "Level 1", (int)(hudH * 0.5f), Ink); _levelTxt.rectTransform.Stretch(6);
RopeWordGame.View.cs:119 / void BuildView()	var title = HzUI.Label("Title", _layer, "Rope Riddle", 58, Ink);
RopeWordGame.View.cs:172 / void BuildView()	_clueTxt = HzUI.Label("Cat", _clueCard.transform, "", 54, Ink);
RopeWordGame.View.cs:209 / void Pill(RectTransform parent, float x, float y, float w, float h, Color iconColor, out Text label)	label = HzUI.Label("V", p.transform, "0", (int)(h * 0.56f), Ink);

RopeWordGame.View.cs:255 / float BuildSlots(float cx, float top, float availW, float maxTile)	s.line = HzUI.Panel("Line", s.rt, InkSoft, 4);
RopeWordGame.View.cs:257 / float BuildSlots(float cx, float top, float availW, float maxTile)	s.txt = HzUI.Label("L", s.rt, "", (int)(t * 0.64f), Ink);
RopeWordGame.View.cs:277 / float BuildSlots(float cx, float top, float availW, float maxTile)	k.txt = HzUI.Label("K", img.transform, char.ToUpperInvariant(c).ToString(), (int)(h * 0.5f), Ink);
RopeWordGame.View.cs:301 / void BuildHelperBar(Rect bar)	var name = HzUI.Label("Name", ui.rt, d.Name, 26, Ink); name.rectTransform.Place(new Vector2(0.5f, 0), new Vector2(slotW, 34), new Vector2(0, -24));
RopeWordGame.View.cs:308 / void BuildHelperBar(Rect bar)	var pr = HzUI.Panel("Price", ui.rt, Ink, 18); pr.rectTransform.Place(new Vector2(1, 1), new Vector2(84, 40), new Vector2(-14, -2));
RopeWordGame.View.cs:316 / void BuildHelperBar(Rect bar)	ui.lockTxt = HzUI.Label("L", lp.transform, "LV " + d.Unlock, 24, Ink); ui.lockTxt.rectTransform.Stretch();
RopeWordGame.View.cs:423 / void BuildScene(Rect r)	_timerTxt = HzUI.Label("T", _timerPill.transform, "0:00", 40, Ink); _timerTxt.rectTransform.Stretch(); _timerTxt.rectTransform.offsetMin = new Vector2(56, 0);
RopeWordGame.View.cs:566 / void RefreshClue(bool animate)	_clueTxt.color = vis ? Ink : InkSoft;
RopeWordGame.View.cs:585 / void RefreshTimer()	_timerTxt.color = _bonusLost ? InkSoft : sec <= 10 ? Bad : Ink;
RopeWordGame.View.cs:594 / void StyleSlot(int i, bool lostShow)	t.txt.color = Ink;
RopeWordGame.View.cs:595 / void StyleSlot(int i, bool lostShow)	t.line.color = shown ? Good : (lostShow ? Bad : InkSoft);
RopeWordGame.View.cs:601 / void StyleKey(Key k)	if (R.Removed(k.c)) { k.img.color = new Color(KeyUsed.r, KeyUsed.g, KeyUsed.b, 0.45f); k.txt.color = new Color(InkSoft.r, InkSoft.g, InkSoft.b, 0.35f); return; }
RopeWordGame.View.cs:602 / void StyleKey(Key k)	if (!R.Has(k.c)) { k.img.color = KeyBg; k.txt.color = Ink; return; }

Source	Exact current code
`RopeWordGame.cs:23 / class/static declarations`	`new HelperDef { Id = "reveal", Name = "Reveal", Sprite = "helper_reveal", Glyph = HzIcons.Search, Col = "#19B5A5", Unlock = 3, Cost = 10, Free = 3,`
`RopeWordGame.cs:24 / class/static declarations`	`Desc = "Uncovers the first hidden letter of the answer.", Guide = "Tap <color=#FFD166>Reveal</color> to uncover a letter. This one is free!" },`
`RopeWordGame.cs:25 / class/static declarations`	`new HelperDef { Id = "snip", Name = "Snip", Sprite = "helper_remove", Glyph = "", Col = "#FF7A45", Unlock = 6, Cost = 15, Free = 2,`
`RopeWordGame.cs:26 / class/static declarations`	`Desc = "Cuts 3 wrong letters off the keyboard. No strands lost!", Guide = "Tap <color=#FFD166>Snip</color> to cut 3 wrong letters off the keyboard." },`
`RopeWordGame.cs:27 / class/static declarations`	`new HelperDef { Id = "lantern", Name = "Lantern", Sprite = "helper_vowels", Glyph = "", Col = "#7A5BA8", Unlock = 10, Cost = 20, Free = 2,`
`RopeWordGame.cs:28 / class/static declarations`	`Desc = "Lights up the vowels: the right ones appear, the wrong ones go dark.", Guide = "Tap the <color=#FFD166>Lantern</color> to light up every vowel." },`
`RopeWordGame.cs:29 / class/static declarations`	`new HelperDef { Id = "rope", Name = "Spare Rope", Sprite = "helper_life", Glyph = HzIcons.Heart, Col = "#E2553F", Unlock = 15, Cost = 25, Free = 1,`
`RopeWordGame.cs:30 / class/static declarations`	`Desc = "Ties one extra strand onto the rope: one more mistake allowed.", Guide = "Tap <color=#FFD166>Spare Rope</color> to tie on an extra strand." },`
`RopeWordGame.UI.cs:88 / void ShowUnlock(List<int> list, int k)`	`CardText(m, d.Desc + (d.Free > 0 ? $"\n<color=#2FA66A>+{d.Free} free to try! </color>" : ""), 660, 690, 150, 38, HzUI.Hex("#5A4636"));`
`RopeWordGame.UI.cs:144 / void EndGuide(int h)`	`Toast("Nice! " + Defs[h].Name + " is ready whenever you need it.", HzUI.Hex("#7CFFB2"), 1.8f);`

RopeWordGame.UI.cs:252 / void WordTiles(ModalInfo m, float y, bool revealAll)	var p = HzUI.Panel("T", m.card, got ? Good : revealAll ? Bad : HzUI.Hex("#D8CBB8"), (int)(size * 0.2f));
RopeWordGame.UI.cs:303 / void ShowWin(int level, int stars, int gems, bool timedBonus)	CardText(m, $"+{gems} gems" + (timedBonus ? "<color=#FF7A45>(speed bonus!)</color>" : ""), 745, 600, 46, 36, Gem);
RopeWordGame.UI.cs:374 / void ShowSettingsModal()	sound = CardButton(m, SoundLabel(), HzUI.Hex("#4C9EEB"), 190, 560, 112, null);
RopeWordGame.UI.cs:376 / void ShowSettingsModal()	music = CardButton(m, MusicLabel(), HzUI.Hex("#4C9EEB"), 330, 560, 112, null);
RopeWordGame.View.cs:14 / class/static declarations	static readonly Color BgTop = HzUI.Hex("#FFF1DE"), BgBot = HzUI.Hex("#FFC99E");
RopeWordGame.View.cs:15 / class/static declarations	static readonly Color SkyTop = HzUI.Hex("#FFE3C2"), SkyBot = HzUI.Hex("#FFB38A");
RopeWordGame.View.cs:16 / class/static declarations	static readonly Color Cliff = HzUI.Hex("#D9895B"), CliffDark = HzUI.Hex("#B4643E");
RopeWordGame.View.cs:17 / class/static declarations	static readonly Color Wood = HzUI.Hex("#8A5634"), WoodDark = HzUI.Hex("#5E3820"), WoodLight = HzUI.Hex("#B07A4F");
RopeWordGame.View.cs:18 / class/static declarations	static readonly Color Card = HzUI.Hex("#FFF7EA"), Ink = HzUI.Hex("#23384A"), InkSoft = HzUI.Hex("#6B7C8A");
RopeWordGame.View.cs:19 / class/static declarations	static readonly Color Good = HzUI.Hex("#2FA66A"), Bad = HzUI.Hex("#E2553F"), Gem = HzUI.Hex("#19B5A5");
RopeWordGame.View.cs:20 / class/static declarations	static readonly Color Coral = HzUI.Hex("#FF7A45"), Sun = HzUI.Hex("#FFD166"), Plum = HzUI.Hex("#7A5BA8");
RopeWordGame.View.cs:21 / class/static declarations	static readonly Color KeyBg = HzUI.Hex("#FFFFFF"), KeyUsed = HzUI.Hex("#E6D9C8");

RopeWordGame.View.cs:22 / class/static declarations	static readonly string[] StrandHex = { "#F2D49B", "#E8C07A", "#D9A85E", "#F0CB86", "#E3B46C", "#D19A4E", "#EBC88E", "#F5DCA8", "#DDB070" };
RopeWordGame.View.cs:28 / static Color WaterOf(string scene)	case "jungle": return HzUI.Hex("#2E9C8F");
RopeWordGame.View.cs:29 / static Color WaterOf(string scene)	case "snow": return HzUI.Hex("#5AA9D6");
RopeWordGame.View.cs:30 / static Color WaterOf(string scene)	case "night": return HzUI.Hex("#2B4C8C");
RopeWordGame.View.cs:31 / static Color WaterOf(string scene)	case "autumn": return HzUI.Hex("#3B86B0");
RopeWordGame.View.cs:32 / static Color WaterOf(string scene)	case "coast": return HzUI.Hex("#1FA2B8");
RopeWordGame.View.cs:33 / static Color WaterOf(string scene)	default: return HzUI.Hex("#2E8FB5");
RopeWordGame.View.cs:332 / void RefreshHelpers()	ui.priceTxt.color = Gems >= Defs[h].Cost ? Color.white : HzUI.Hex("#FF9A8A");
RopeWordGame.View.cs:385 / void BuildScene(Rect r)	Shape("Bracket", S, HzUI.Hex("#4A3A30"), 8, ax, beamY, 46 * s, _beamH + 12 * s);
RopeWordGame.View.cs:386 / void BuildScene(Rect r)	var pulley = HzUI.Img("Pulley", S, HzUI.Circle(32), HzUI.Hex("#6B5A4E")); pulley.raycastTarget = false; L(pulley.rectTransform, ax, beamY + _beamH * 0.55f, 34 * s, 34 * s);
RopeWordGame.View.cs:513 / void SetPose(Pose p, bool force = false)	Color skin = HzUI.Hex("#F5C99B"), shirt = HzUI.Hex("#2BB3A3"), pants = HzUI.Hex("#3D4F7A"), dark = HzUI.Hex("#2A2A35");
RopeWordGame.View.cs:514 / void SetPose(Pose p, bool force = false)	Part("Pack", C, HzUI.Hex("#8E5A3C"), 12, -32 * s, 152 * s, 38 * s, 62 * s);
RopeWordGame.View.cs:533 / void SetPose(Pose p, bool force = false)	var ring = HzUI.Img("Ring", C, HzUI.RoundRect(40, 16), HzUI.Hex("#FF5A4E")); ring.raycastTarget = false; ring.type = Image.Type.Sliced; ring.preserveAspect = false;
RopeWordGame.View.cs:596 / void StyleSlot(int i, bool lostShow)	t.bg.color = shown ? HzUI.Hex("#EAF8EF") : (lostShow ? HzUI.Hex("#FBE3DE") : Card);

CHAPTER 10

Reskin recipe: visible menu and HUD text

Open `Scripts/RopeWordGame.cs`, find the displayed literal `"Rope Riddle"`, and replace its displayed value with `"Rope Riddle Edition"`. Keep hierarchy names and technical IDs unchanged. In HzUI.Label/Button calls the first string may be an object name; change the displayed text argument. Read the method references below to distinguish construction from later refresh assignments.

Preserve interpolation braces, invariant number formatting, ON/OFF updates and line breaks. Update both initial and refresh paths, portrait and landscape. Exact helper names/unlocks/charges: Scripts/RopeWordGame.cs; help and effect descriptions: Scripts/RopeWordGame.cs. Level names/hints remain separate JSON content. If a title appears in art, Title property or another screen, update those visible uses deliberately without renaming GameId, namespace, resources or save keys.

Hard-coded game strings do not automatically use shared localization. To translate the interface, route every display assignment through your translation table. Dictionaries, word banks and theme pools are independent gameplay assets and need separate validation. Test help, settings, results, tutorial and helper introductions after a fresh launch.

Source	Exact current code
`RopeWordGame.cs:23 / class/static declarations`	`new HelperDef { Id = "reveal", Name = "Reveal", Sprite = "helper_reveal", Glyph = HzIcons.Search, Col = "#19B5A5", Unlock = 3, Cost = 10, Free = 3,`
`RopeWordGame.cs:24 / class/static declarations`	`Desc = "Uncovers the first hidden letter of the answer.", Guide = "Tap <color=#FFD166>Reveal</color> to uncover a letter. This one is free!" },`
`RopeWordGame.cs:25 / class/static declarations`	`new HelperDef { Id = "snip", Name = "Snip", Sprite = "helper_remove", Glyph = "□", Col = "#FF7A45", Unlock = 6, Cost = 15, Free = 2,`
`RopeWordGame.cs:26 / class/static declarations`	`Desc = "Cuts 3 wrong letters off the keyboard. No strands lost!", Guide = "Tap <color=#FFD166>Snip</color> to cut 3 wrong letters off the keyboard." },`
`RopeWordGame.cs:27 / class/static declarations`	`new HelperDef { Id = "lantern", Name = "Lantern", Sprite = "helper_vowels", Glyph = "□", Col = "#7A5BA8", Unlock = 10, Cost = 20, Free = 2,`
`RopeWordGame.cs:28 / class/static declarations`	`Desc = "Lights up the vowels: the right ones appear, the wrong ones go dark.", Guide = "Tap the <color=#FFD166>Lantern</color> to light up every vowel." },`

`RopeWordGame.cs:29 / class/static declarations`	`new HelperDef { Id = "rope", Name = "Spare Rope", Sprite = "helper_life", Glyph = HzIcons.Heart, Col = "#E2553F", Unlock = 15, Cost = 25, Free = 1,`
`RopeWordGame.cs:30 / class/static declarations`	`Desc = "Ties one extra strand onto the rope: one more mistake allowed.", Guide = "Tap <color=#FFD166>Spare Rope</color> to tie on an extra strand." },`
`RopeWordGame.cs:240 / void OnMiss()`	`if (m == 3 && Unlocked(HReveal) && !Flag("nudged")) Bubble("Stuck? A helper can save the day!", Gem, 1.8f, _rHelpers);`
`RopeWordGame.cs:265 / void OnHelper(int h)`	`Bubble(d.Name + " unlocks at level " + d.Unlock, Ink, 1.5f, HelperRect(h));`
`RopeWordGame.cs:270 / void OnHelper(int h)`	`if (!CanUse(h, out var why)) { HzAudio.Play("locked", 0.5f); HzTween.Shake(_helperUI[h].rt, 8, 0.25f); Bubble(why, Ink, 1.4f, HelperRect(h)); return; }`
`RopeWordGame.cs:424 / public override string DebugCommand(string cmd)`	`case "helper": if (p.Length > 1 && int.TryParse(p[1], out var hi) && hi >= 0 && hi < 4) { OnHelper(hi); return "charges=" + Charges(hi) + " gems=" + Gems + " masked=" + R.Masked(); } return "usage: helper 0-3";`
`RopeWordGame.cs:425 / public override string DebugCommand(string cmd)`	`case "unlockall": for (int i = 0; i < 4; i++) { SetFlag("unl_" + Defs[i].Id); SetFlag("guide_" + Defs[i].Id); } RefreshHelpers(); return "ok";`
`RopeWordGame.UI.cs:61 / void LevelHint()`	`CardButton(m, "Got it!", Good, 450, 420, 112, () => { _enterAction = ok; ok(); }, 52);`
`RopeWordGame.UI.cs:97 / void ShowUnlock(List<int> list, int k)`	`CardButton(m, k + 1 < list.Count ? "Next" : "Try it!", Good, 820, 460, 116, () => { _enterAction = go; go(); }, 56);`
`RopeWordGame.UI.cs:144 / void EndGuide(int h)`	`Toast("Nice! " + Defs[h].Name + " is ready whenever you need it.", HzUI.Hex("#7CFFB2"), 1.8f);`
`RopeWordGame.UI.cs:157 / void Bubble(string text, Color color, float secs, Rect anchor, RectTransform parent = null)`	`var lbl = HzUI.Label("T", b.transform, text, 38, Color.white); lbl.supportRichText = true; lbl.rectTransform.Stretch(18);`

`RopeWordGame.UI.cs:184 / void Bubble(string text, Color color, float secs, Rect anchor, RectTransform parent = null)`	`var tl = HzUI.Label("Title", ribbon.transform, title, 62, Color.white); tl.rectTransform.Stretch(6);`
`RopeWordGame.UI.cs:230 / void CloseAllModals()`	`var t = HzUI.Label("Txt", m.card, text, size, color, TextAnchor.MiddleCenter, display);`
`RopeWordGame.UI.cs:241 / static void AdTag(Transform parent)`	`HzUI.Label("t", tag.transform, "AD", 26, Color.white).rectTransform.Stretch();`
`RopeWordGame.UI.cs:255 / void WordTiles(ModalInfo m, float y, bool revealAll)`	`HzUI.Label("L", p.transform, ch, (int)(size * 0.6f), Color.white).rectTransform.Stretch();`
`RopeWordGame.UI.cs:308 / void ShowWin(int level, int stars, int gems, bool timedBonus)`	`CardButton(m, "Next level", Good, 850, 560, 124, () => { _enterAction = next; next(); }, 58);`
`RopeWordGame.UI.cs:309 / void ShowWin(int level, int stars, int gems, bool timedBonus)`	`var home = HzUI.Button("Home", m.card, "", Plum, Color.white, () => { _enterAction = null; CloseModal(m); NewRound(); ShowMenu(); }, 40, 30);`
`RopeWordGame.UI.cs:329 / void ShowLose()`	`var ad = CardButton(m, "Extra rope: +2 strands", Gem, y, 600, 124, null, 48);`
`RopeWordGame.UI.cs:338 / void ShowLose()`	`CardButton(m, "Try again", Good, y, 560, 120, () => { _enterAction = retry; retry(); }, 54);`
`RopeWordGame.UI.cs:339 / void ShowLose()`	`var home = HzUI.Button("Home", m.card, "", Plum, Color.white, () => { _enterAction = null; CloseModal(m); NewRound(); ShowMenu(); }, 40, 30);`
`RopeWordGame.UI.cs:351 / void OfferGems(int cost)`	`var ad = CardButton(m, "Watch ad: +15 gems", Good, 440, 560, 116, null, 46);`
`RopeWordGame.UI.cs:364 / void OfferGems(int cost)`	`CardButton(m, "No, thanks", InkSoft, 570, 420, 96, () => CloseModal(m), 42);`
`RopeWordGame.UI.cs:374 / void ShowSettingsModal()`	`sound = CardButton(m, SoundLabel(), HzUI.Hex("#4C9EEB"), 190, 560, 112, null);`
`RopeWordGame.UI.cs:375 / void ShowSettingsModal()`	`sound.onClick.AddListener(() => { HzAudio.Muted = !HzAudio.Muted; sound.SetText(SoundLabel()); HzAudio.Play("toggle", 0.5f); });`
`RopeWordGame.UI.cs:376 / void ShowSettingsModal()`	`music = CardButton(m, MusicLabel(), HzUI.Hex("#4C9EEB"), 330, 560, 112, null);`

RopeWordGame.UI.cs:377 / void ShowSettingsModal()	music.onClick.AddListener(() => { HzAudio.MusicMuted = !HzAudio.MusicMuted; music.SetText(MusicLabel()); HzAudio.Play("toggle", 0.5f); });
RopeWordGame.UI.cs:380 / void ShowSettingsModal()	CardButton(m, "Restart level", Coral, 470, 560, 112, () =>
RopeWordGame.UI.cs:386 / void ShowSettingsModal()	CardButton(m, "Home", Plum, 610, 560, 112, () => { CloseModal(m); ShowMenu(); });
RopeWordGame.UI.cs:387 / void ShowSettingsModal()	CardButton(m, "Resume", Good, 770, 560, 120, () => CloseModal(m), 56);
RopeWordGame.UI.cs:389 / void ShowSettingsModal()	else CardButton(m, "Close", Good, 490, 560, 120, () => CloseModal(m), 56);
RopeWordGame.UI.cs:391 / static string SoundLabel() => HzAudio.Muted ? "Sound: OFF" : "Sound: ON";	static string SoundLabel() => HzAudio.Muted ? "Sound: OFF" : "Sound: ON";
RopeWordGame.UI.cs:392 / static string MusicLabel() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";	static string MusicLabel() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";
RopeWordGame.UI.cs:440 / void ShowMenu()	var t1 = HzUI.Label("T", sign.transform, "Rope Riddle", portrait ? 120 : 130, Ink); t1.rectTransform.Stretch(); t1.rectTransform.anchoredPosition = new Vector2(0, 18);
RopeWordGame.UI.cs:442 / void ShowMenu()	var t2 = HzUI.Label("S", sign.transform, "Guess the word. Keep the climber on the rope!", 34, InkSoft, TextAnchor.MiddleCenter, false);
RopeWordGame.UI.cs:448 / void ShowMenu()	var play = HzUI.Button("Play", _menu, "Play", Good, Color.white, () =>
RopeWordGame.UI.cs:455 / void ShowMenu()	HzUI.Label("Lv", lvTag.transform, "Level " + Level, 44, Ink).rectTransform.Stretch();
RopeWordGame.UI.cs:462 / void ShowMenu()	var sT = HzUI.Label("V", sp.transform, TotalStars.ToString(), 54, Ink); sT.rectTransform.Stretch(); sT.rectTransform.offsetMin = new Vector2(90, 0);

`RopeWordGame.UI.cs:529 / void Dim(float x, float y, float w, float h) { if (w <= 0 || h <= 0) return; var d = HzUI.Panel("Dim", _tut, dark, 0); d.rectTransform.Place(new Vector2(0, 1), new Vector2(w, h), new Vector2(x, -y), new Vector2(0, 1)); }`	`var ct = HzUI.Label("T", cap.transform, TutText[_tutStep], 42, Ink); ct.rectTransform.Stretch(26);`
`RopeWordGame.UI.cs:530 / void Dim(float x, float y, float w, float h) { if (w <= 0 || h <= 0) return; var d = HzUI.Panel("Dim", _tut, dark, 0); d.rectTransform.Place(new Vector2(0, 1), new Vector2(w, h), new Vector2(x, -y), new Vector2(0, 1)); }`	`var step = HzUI.Label("N", cap.transform, (_tutStep + 1) + "/" + TutText.Length, 26, InkSoft, TextAnchor.UpperRight, false);`
`RopeWordGame.UI.cs:532 / void Dim(float x, float y, float w, float h) { if (w <= 0 || h <= 0) return; var d = HzUI.Panel("Dim", _tut, dark, 0); d.rectTransform.Place(new Vector2(0, 1), new Vector2(w, h), new Vector2(x, -y), new Vector2(0, 1)); }`	`var tap = HzUI.Label("Tap", _tut, "Tap to continue", 36, Color.white, TextAnchor.MiddleCenter, false);`
`RopeWordGame.View.cs:106 / void BuildView()`	`var home = HzIcons.IconButton(_layer, HzIcons.Home, Plum, () => { if (!Overlaid && !_ended) { HzAudio.Play("popup_close", 0.4f); ShowMenu(); } }, hudH);`
`RopeWordGame.View.cs:111 / void BuildView()`	`_levelTxt = HzUI.Label("Lv", lvPill.transform, "Level 1", (int)(hudH * 0.5f), Ink); _levelTxt.rectTransform.Stretch(6);`
`RopeWordGame.View.cs:119 / void BuildView()`	`var title = HzUI.Label("Title", _layer, "Rope Riddle", 58, Ink);`
`RopeWordGame.View.cs:171 / void BuildView()`	`_clueTag = HzUI.Label("t", tag.transform, "CATEGORY", 26, Color.white); _clueTag.rectTransform.Stretch();`
`RopeWordGame.View.cs:172 / void BuildView()`	`_clueTxt = HzUI.Label("Cat", _clueCard.transform, "", 54, Ink);`
`RopeWordGame.View.cs:209 / void Pill(RectTransform parent, float x, float y, float w, float h, Color iconColor, out Text label)`	`label = HzUI.Label("V", p.transform, "0", (int)(h * 0.56f), Ink);`
`RopeWordGame.View.cs:257 / float BuildSlots(float cx, float top, float availW, float maxTile)`	`s.txt = HzUI.Label("L", s.rt, "", (int)(t * 0.64f), Ink);`

RopeWordGame.View.cs:277 / float BuildSlots(float cx, float top, float availW, float maxTile)	k.txt = HzUI.Label("K", img.transform, char.ToUpperInvariant(c).ToString(), (int)(h * 0.5f), Ink);
RopeWordGame.View.cs:286 / void BuildHelperBar(Rect bar)	var panel = HzUI.Panel("HelperBar", _layer, new Color(1, 1, 1, 0.55f), 40);
RopeWordGame.View.cs:296 / void BuildHelperBar(Rect bar)	var b = HzUI.Button("Helper_" + d.Id, _layer, null, HzUI.Hex(d.Col), Color.white, () => OnHelper(hh), 40, (int)(size * 0.28f));
RopeWordGame.View.cs:301 / void BuildHelperBar(Rect bar)	var name = HzUI.Label("Name", ui.rt, d.Name, 26, Ink); name.rectTransform.Place(new Vector2(0.5f, 0), new Vector2(slotW, 34), new Vector2(0, -24));
RopeWordGame.View.cs:305 / void BuildHelperBar(Rect bar)	ui.badgeTxt = HzUI.Label("N", bd.transform, "0", 30, Color.white); ui.badgeTxt.rectTransform.Stretch();
RopeWordGame.View.cs:310 / void BuildHelperBar(Rect bar)	ui.priceTxt = HzUI.Label("P", pr.transform, d.Cost.ToString(), 28, Color.white); ui.priceTxt.rectTransform.Stretch(); ui.priceTxt.rectTransform.offsetMin = new Vector2(34, 0);
RopeWordGame.View.cs:316 / void BuildHelperBar(Rect bar)	ui.lockTxt = HzUI.Label("L", lp.transform, "LV" + d.Unlock, 24, Ink); ui.lockTxt.rectTransform.Stretch();
RopeWordGame.View.cs:358 / void BuildScene(Rect r)	string sceneName = _data != null ? _data.scene : "canyon";
RopeWordGame.View.cs:423 / void BuildScene(Rect r)	_timerTxt = HzUI.Label("T", _timerPill.transform, "0:00", 40, Ink); _timerTxt.rectTransform.Stretch(); _timerTxt.rectTransform.offsetMin = new Vector2(56, 0);
RopeWordGame.View.cs:543 / void RefreshAll()	_levelTxt.text = "Level " + Level;
RopeWordGame.View.cs:565 / void RefreshClue(bool animate)	_clueTxt.text = vis ? R.category : "? ? ?";
RopeWordGame.View.cs:567 / void RefreshClue(bool animate)	_clueTag.text = R.hidden && !vis ? "MYSTERY CLUE" : "CATEGORY";
RopeWordGame.View.cs:583 / void RefreshTimer()	_timerTxt.text = _bonusLost ? "Bonus lost" : (sec / 60) + ":" + (sec % 60).ToString("00");

`RopeWordGame.View.cs:593 / void StyleSlot(int i, bool lostShow)`	`t.txt.text = shown ? char.ToUpperInvariant(R.answer[i]).ToString() : "";`

CHAPTER 11

Artwork, fonts, audio and licences

Replace game images under Resources/ropeword through Unity while retaining resource keys, .meta GUIDs, Sprite/UI import settings, alpha, pivots and slicing. Procedural graphics in the art/view sources may need separate colour changes. Replacing one PNG does not change every fallback. Keep third-party font and other licence notices with the corresponding shipped assets.

Effects are under Resources/ropeword/Sfx. Preserve each actual load key and importer settings when replacing sounds. Supplied music: Resources/ropeword/Music/canyon_breeze.ogg. Preserve that load key and verify loop/compression and the independent Music toggle. Kit UI uses supplied fonts; preserve actual licence notices and verify new glyphs.

CHAPTER 12

Local saves, reset and migration

HzSave wraps PlayerPrefs and flushes local state on focus loss/quit. WebGL persistence uses browser storage tied to the hosting origin. Clearing site data or changing origin starts a new local save; this is not cloud synchronization. Inspect HzSave/PlayerPrefs calls in the actual game source before altering progression, stars, helper charges or tutorial flags.

Verify fresh launch, a completed arena, helper use, sound options, focus loss and restart. Use a dedicated test profile for resets. Keep technical save keys stable during colour/text reskins. If changing schema or keys, implement and test a migration rather than silently discarding existing users' progress.

CHAPTER 13

Advertising integration and configuration

Asset: `Assets/Szekipapa77Ads/Resources/AdsSettings.asset`. Inspect backend, useGoogleTestIds and missingSdkFallback in the publisher-owned Ads configuration. Buyer projects include adapter/integration code and exclude external provider SDK binaries. WebGL and a missing-provider fallback simulate ads; these are not live commercial impressions.

For real Android ads install the supported provider SDK from its official source into your working project, enter your own app/placement IDs and use test ads first. Verify banner/interstitial/rewarded callbacks on supported hardware. For production use your own IDs and disable simulated missing-SDK fallback. Build tooling rejects unsafe production fallback settings. No IAP, remote backend or HLK is included in this publisher edition.

CHAPTER 14

Android build and release

Install Unity 6000.0.62f1 Android modules through Hub. Select Android, run Setup and review application ID `com.szekipapa77.ropeword`, version, scene orientation and signing. The build command writes `Builds/Android/ropeword.apk`. Confirm BuildResult.Succeeded and the actual output file; intermediate report totals are not the final APK size.

Use your own keystore/signing and distribution settings. A test-signed demo is not a signed store release. A provider-free build simulates ads; a provider test-demo does not prove production ad-account configuration. Test real hardware touch, safe areas, focus/audio, save/relaunch and reward callbacks. Keep passwords, keystores and private service credentials out of buyer packages.

CHAPTER 15

WebGL build and hosting

Install matching WebGL support. Run Setup and Build WebGL; output is `Builds/WebGL/ropeword/`. Serve index.html with all loader/framework/data/WASM/StreamingAssets files over HTTP(S), not file://. Match compression, Content-Encoding and MIME types to the host. Outer page sizing and actual scene orientation are different settings.

The publisher's playable demo is hosted on HyperCodeStore. Upload the actual runtime files rather than merely changing a URL. Test a fresh anonymous load, real pointer controls, audio after interaction, saved progress/reload and phone/desktop sizing. Rebuild and rehost when code/assets change. A successful HTTP page alone is not gameplay verification.

CHAPTER 16

Troubleshooting

Compile errors: restore matching packages and the full private Kit/Ads layer; only one game Kit belongs in a project. **Pink/invisible rendering:** run Setup and inspect Graphics and every Quality URP assignment. **No input:** classic input handling must be enabled; check canvas focus and blocking modals. **Missing resources:** restore exact paths, .meta GUIDs and importer settings.

Edits do not appear: stop Play, Save in the level editor, reload the canonical bank and check current saved level. **Text reverts:** update construction and refresh assignments from the exact table. **No live ads:** provider binaries are deliberately excluded; inspect your installed provider and fallback mode. **Scene reload exception:** retain owner-lifetime and generation guards on delayed callbacks. **WebGL fails:** inspect browser Console, compression headers and missing payloads.

CHAPTER 17

Support and final release checklist

Publisher: Szekipapa77. Support: szekipapa77@gmail.com. Include game title, exact Unity version, platform, first Console error and reproduction steps. Never send passwords/tokens.

1. Clean-import the standalone project and native package.
2. Play both scenes and replace a scene near startup.
3. Review tutorial, normal completion/retry and every helper activation plus effect.
4. Validate editor changes and test reskin text/colours in both layouts.
5. Rebuild affected Android/WebGL outputs.
6. Verify complete public APK bytes, actual browser demo, documentation and video links.
7. Keep licences and exclude all internal prompts, planning/capture/build/QA tooling from buyers.